1 MAN COMPANY

Agentic Engineering

10 DOs & Don'ts

How to Run Full Company Output with AI Agents

Startup4Chinese Meetup 2026

10	Key Practices
2	Core AI Tools
1	1 Person = Full Team

github.com/bryanzk/MyCodexEnv

You are one person,

but you need to do the work of 5.

This isn't a complaint — it's the reality of a 1 Man Company.

So, can this actually be done?

Roles one person must play

Product / PM	Requirements · Prioritization
BA	Business Modeling · Acceptance Criteria
Engineer / Dev	Architecture · Coding · Testing
QA	Quality Control · Regression Testing
DevOps	Deploy · Environment · Release

Harvard Business Review · 2026 | 200 Engineers · 9 Months

Used AI, yet ended up with more work.

01

Speed up, more tasks pile up

Management sees productivity gains and immediately assigns more work

02

Work hours self-extend

Not required — you just feel "I can do a little more"

03

Multitasking threads, never truly finishing

Managing multiple AI threads simultaneously, attention always fragmented

The problem is not AI — it's how you use it.

Most people treat AI as an assistant — resulting in AI amplifying existing workload

Agentic Engineering treats AI as a team — redefining your relationship with work

Not doing more — choosing to do the right things

Norbert Wiener · 1948 · Cybernetics · from κυβερνήτης — helmsman

Three times, the same pattern.

Each time, someone closed the feedback loop at a new level.

1780s

Watt's Governor

Before: Workers manually adjusted valves

Now: **Workers design governors**

2010s

Kubernetes

Declarative Infra

Before: Engineers manually restarted services

Now: **Engineers write specification files**

2020s

AI Agent

Harness Engineering

Before: Engineers handwrite code

Now: **Engineers design the Harness**

You no longer turn the valve by hand — you steer. Work hasn't disappeared — it has elevated to a higher level.

“Any knowledge that can be methodologized will be replaced by AI.

The 1 Man Company

One person. Full team output.

C

Claude

Product / BA

Decompose business requirements
Clarify scope and assumptions
Deliver MVP solution

Requirements · Scope · MVP

C

Codex

Engineering Team

Plan tasks
Test-Driven Development
Close loop to Git

Plan · TDD · Ship

S

Skill

Engineering Culture

Codify best practices
Reuse across projects
Standardize team practices

Reuse · Scale · Standards

G

GitHub

DevOps / Sync

Multi-machine environment sync
Version control
Continuous delivery

Sync · Version · Deploy

Why "Now" Is the Critical Inflection Point

The codebase always had feedback loops — they just never closed at the most critical layer. Until now.

Syntax Layer	**Compiler** Compiler	*Does it compile?*	70+ years
Behavior Layer	**Test Suite** Test Suite	*Does it pass tests?*	30+ years
Style Layer	**Linter** Linter	*Does it conform to standards?*	40+ years
Judgment Layer	**LLM** LLM / Agent	*Is this the right direction?*	🚀 **Now**

Your new role: define "correct" at the Judgment Layer —Architecture docs, custom Lint rules, golden examples.

Framework: Claude → Codex → Ship

End-to-end pipeline: Define → Build → Deliver

01 Claude

What

Problem Definition

- **3-Element Constraint** 3-element prompt
- **MVP output**
- **Requirement decomposition**
- **Clarify boundaries**

02 Codex 5.3

How

Engineering Execution

- **Plan Mode** Plan before code
- **Test-Driven Development (TDD)**
- **Root cause analysis**
- **Git-closed loop**

03 Review + Git

Ship

Delivery & Acceptance

- **Code review Skill**
- **Acceptance testing**
- **Doc sync**
- **Merge & deploy**

“The limits of my language mean the limits of my world.

— Ludwig Wittgenstein, Tractatus Logico-Philosophicus, 1921

Chapter 1

Rethink How You Ask

Most people's first mistake using AI isn't choosing the wrong tool — it's asking the wrong question. Master the 3-element prompt to make AI truly work for you.

01 The 3-Element Prompt: Goal · Context · Best Practices

DO

- Always provide 3 elements: Goal (Why) / Context / Constraints (How)
- Reference industry best practices as the constraint baseline
- Clearly define your business context and system boundaries
- Result: converge to a business-specific solution on the first try

DON'T

Have you ever asked AI "how should I do this?" and got a generic answer with no relevance to your business?

- No context = AI can only give generic frameworks
- AI can't know your system architecture or business constraints
- Result: multiple rounds of back-and-forth waste tokens and time

02 Meta-Prompt: Refine Your Question Before Asking It

DO

- Use a meta-prompt to optimize your initial prompt before formally asking
- 4 dimensions: clarity / expertise / structure / model adaptability
- Make AI "examine the question" before "answering the question"
- Result: from ChatGPT outputting 24 items → Claude focused to 8

DON'T

Have you ever received 24 AI suggestions, each "somewhat useful," but none truly effective for you?

- Unoptimized prompts produce broad, scattered results
- 24 acceptance criteria: looks complete, but hard to execute
- Can't target your specific business context

02 Meta-Prompt Template

Please do not execute the following task. Your current role is that of a world-class prompt engineering expert. Carefully read the [prompt] I provide, identify the top 3 experts in the relevant industry domain(s), form an expert committee, and critically optimize it across four dimensions: clarity, professionalism, structure, and model adaptability. Then, have the expert committee conduct multiple rounds of debate to reach a conclusion. Output only the optimized prompt content, and wrap it in triple backticks.

[prompt]

03 MVP First: Focus the AI, Avoid Over-Generation

DO

- When requirements are unclear, explicitly ask for MVP version output
- Key phrase: "Give only the minimum viable version, focus on 3 core constraints"
- Rapidly converge complex discussions into directly executable task packages
- Result: avoid the trap of docs that "look comprehensive but won't run"

DON'T

Have you asked AI for a solution and received a comprehensive doc with no idea where to start executing?

- Over-complete output = massive and hard to understand
- 24 acceptance criteria that are hard to implement
- High verification cost actually slows engineering delivery

Chapter 2

Make Engineering Real

Choosing the right tools is step one. Real engineering execution depends on how you work. Claude defines problems, Codex plans and executes, Skills ensure quality.

04 Use Claude to Decompose Business Requirements

DO

- Have Claude proactively surface your unstated assumptions
- Workflow: find contradictions → surface them → confirm → then conclude
- Surface hidden decision points you were not aware of
- Result: every conclusion is traceable to a conversation node, not a generic template

DON'T

Have you had Codex write code for three hours, only to find the direction was completely wrong?

- Skipping "clarify boundaries" leads to extremely high rework rates at the implementation layer
- Hidden decisions (e.g., "approximate vs. exact arbitrage") will explode in the code
- Generic answers ≠ your business's answers

05 Plan Mode First — Then Execute

DO

- For complex engineering tasks, use Codex Plan Mode first to plan
- Lock plan → confirm → proceed, minimize back-and-forth
- Codex 5.3 Plan Mode is better at finding root causes across 3+ layers
- Result: complex multi-step tasks land as expected without going off course

DON'T

Have you been chatting and coding with AI, only to realize halfway through that the logic had already drifted?

- Multi-file, multi-step tasks are prone to directional drift
- Execution without a plan = driving at high speed without looking at the road
- Discovering the direction was wrong halfway through → start over

06 Load Community Skills for Automated Best Practices

DO

- Load community Skills (e.g., github.com/obra/superpowers)
- Covers: TDD / system debugging / acceptance validation / code review
- Codex automatically invokes the appropriate Skill for each problem
- Result: engineering best practices ready out of the box, no manual explanation needed each time

DON'T

Have you explained "we use TDD" to the same AI five times, only to have it forget in every new conversation?

- Repetitive engineering actions consume large amounts of context tokens
- “You already taught me that, but I don't remember”
- Unable to accumulate engineering moat; every project starts from zero

07 Build Your Own Reusable Custom Skills

DO

- Codify repeatedly effective prompt patterns into dedicated Skills
- Example: Repo Branch Governance Skill (fully automated branch governance)
- Personal experience → reusable assets across projects
- Result: engineering moat continuously accumulates; team standards become replicable

DON'T

Have you thought "that prompt from the last project was really useful," but searched your chat history and couldn't find it?

- Next project starts from zero, repeating the same mistakes
- “I remember using this approach last time... but I forgot it”
- Personal experience can't be preserved; team can't replicate it

Make your judgment machine-readable.

— Harness Engineering — OpenAI, 2025

Chapter 3

Turn Experience into Assets

A 1 Man Company's moat isn't in tools — it's in the Skill system you accumulate. Make every practice sediment into a reusable capability.

08 Sync Your Skill Environment via GitHub

DO

- Upload your Skill environment to GitHub for plug-and-play across machines
- Reference: github.com/bryanzk/MyCodexEnv
- Switch machines or new environment → one command restores the full configuration
- Result: 1 Man Company with no single point of failure

DON'T

Have you ever switched computers and spent half a day reconfiguring your environment, only to still not get it right?

- Switching machines = start from scratch; this is a single point of failure
- "Only my laptop can run this environment"
- Single point of failure = fatal weakness of a one-person company

09 Visual Output Skills Improve Collaboration

DO

- Use Visual Explainer Skill to output tables, comparison charts, and status matrices
- Example: "For the current code, which items are implemented and which aren't? Give me a table."
- Provide more context so collaborators can quickly grasp decision points
- Result: reduces communication cost, accelerates cross-role decision-making

DON'T

Have you sent a collaborator a 2,000-word analysis, and they replied "Can you give me a table instead?"

- Text lacks structure; collaborators need to reprocess it
- “After reading 2,000 words, what do you actually remember?”
- High-density text = inefficient communication, especially in multi-person collaboration

Auto-Convert Skills Across AI Tools

DO

- superpowers was designed for Claude and can let Codex auto-convert
- Directly reuse community assets with zero manual migration
- “Let AI migrate your AI tool configurations” = Meta-level Agentic
- Result: when upgrading or switching tools, Skill assets continue seamlessly

DON'T

Have you switched from one AI tool to another and found that all your previous work needs to be manually redone?

- This is exactly the repetitive work that Agentic Engineering should eliminate
- If you're doing this manually, you haven't fully leveraged AI
- Tool migration itself can also be Agentic

The 1 Man Company Stack

Right tool, right context, full team output.

Tool	Role	When to Use
Claude	Product / BA	Vague idea → Structured spec
Codex 5.3	Engineering Team	Plan → Build → Test → Merge
Raycast AI	Quick Helper	Quick lookup, no context switch
GitHub	DevOps / Sync	Sync environment across machines
Skill System	Engineering Culture	Personal practices → Team-scale assets

3 Core Principles

The non-negotiable standards for every delivery

V

Verifiable

Every output has a corresponding test or acceptance criterion
Results are traceable; "looks fine" is not acceptable

R

Reproducible

Anyone running the same process on any machine
gets the same result. Environment as code.

S

Shippable

Deliverables meet production-ready standards
Runnable, testable, mergeable, rollback-safe

Agentic Engineering

is not about making you work more —
it's about choosing to do the right things.

You're one person. Now you know how.

Extended Thinking

Can the real world be fully reduced to methodology?

- If not, how much disruption would the non-methodologizable parts cause?
- What are the consequences of letting AI summarize everything with methodology?

No.

Not because "we're not yet smart enough," but because three deeper categories of constraints coexist:

01 Object-Layer Constraints

Reality contains open systems, path dependency, emergence, and reflexivity — none of which can be exhausted by a single static rule.

02 Cognitive-Layer Constraints

Observers can only obtain limited information, and the act of observation itself changes the system.

03 Formal-Layer Constraints

Any sufficiently powerful formal system has parts that are incomplete and undecidable. (Gödel · Turing · Kolmogorov)

Methodology can highly compress reality and improve prediction efficiency, but cannot achieve "full coverage without loss."

Methodology is necessary, but it is always a lossy compression of reality.

The more you treat the compressed result as the whole reality, the greater the systematic error.

Extended Thinking

The Asymmetry of Generation and Verification

P vs NP intuition — empirically reinforced by LLMs (Cobbe et al., 2021)

Generating a correct solution

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Hard

Verifying whether a solution is correct

Easy *relatively speaking*

You don't need to outperform machines in implementation ability——You need to outperform them in judgment ability。

Implications for the AI Era

AI's advantage is on the generation side
Faster, more options, no fatigue, always on

Human's advantage is on the verification side
Judging direction, architecture quality, and trade-off value

Shift in the Center of Capability
From "how to implement" to "what counts as correct implementation"

If AI Is Left to Summarize Everything with Methodology…

Cognitively

· False certainty & overconfidence

· Mistaking the map for the territory

Organizationally

· Metric obsession & template worship

· Degraded capacity to sense anomalies

Socially

· Centralization of interpretive authority

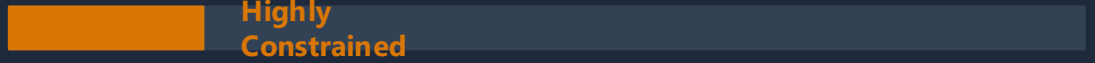
· Complexity flattened → systemic fragility rises

How to Respond

1. All methodologies must be annotated with boundary conditions
2. Systems must retain "unexplained residuals"
3. Look not just at average effects, but also at tail risks
4. Allow reality to correct the framework, rather than forcing reality to conform to the framework

Earth & Nature

Constrained by physical laws long-term



Patterns are predictable; boundaries are fixed

Human Civilization

Trends exist, but full of branching paths

Path dependency + emergence + contingent events

Individual

Future is highly open

Will + choice + creation

The universe sets the boundaries, society shapes the trends, individuals create the path.

Q & A

Thank you!

Questions & discussion welcome

Verifiable

Verifiable

Reproducible

Reproducible

Shippable

Shippable

Resources

Skill Environment	github.com/bryanzk/MyCodexEnv
Superpowers	github.com/obra/superpowers
AI Chinese Tweets	x.com/dotey
HBR Research	hbr.org/2026/02/ai-intensifies